

- *Exceptions are not type-safe.* The type of `failingFn, Int => Int` tells us nothing about the fact that exceptions may occur, and the compiler will certainly not force callers of `failingFn` to make a decision about how to handle those exceptions. If we forget to check for an exception in `failingFn`, this won't be detected until runtime.

Checked exceptions

Java's checked exceptions at least force a decision about whether to handle or reraise an error, but they result in significant boilerplate for callers. More importantly, *they don't work for higher-order functions*, which can't possibly be aware of the specific exceptions that could be raised by their arguments. For example, consider the `map` function we defined for `List`:

```
def map[A,B](l: List[A])(f: A => B): List[B]
```

This function is clearly useful, highly generic, and at odds with the use of checked exceptions—we can't have a version of `map` for every single checked exception that could possibly be thrown by `f`. Even if we wanted to do this, how would `map` even know what exceptions were possible? This is why generic code, even in Java, so often resorts to using `RuntimeException` or some common checked `Exception` type.

We'd like an alternative to exceptions without these drawbacks, but we don't want to lose out on the primary benefit of exceptions: they allow us to *consolidate and centralize error-handling logic*, rather than being forced to distribute this logic throughout our codebase. The technique we use is based on an old idea: instead of throwing an exception, we return a value indicating that an exceptional condition has occurred. This idea might be familiar to anyone who has used return codes in C to handle exceptions. But instead of using error codes, we introduce a new generic type for these “possibly defined values” and use higher-order functions to encapsulate common patterns of handling and propagating errors. Unlike C-style error codes, the error-handling strategy we use is *completely type-safe*, and we get full assistance from the type-checker in forcing us to deal with errors, with a minimum of syntactic noise. We'll see how all of this works shortly.

4.2 Possible alternatives to exceptions

Let's now consider a realistic situation where we might use an exception and look at different approaches we could use instead. Here's an implementation of a function that computes the mean of a list, which is undefined if the list is empty:

```
def mean(xs: Seq[Double]): Double =
  if (xs.isEmpty)
    throw new ArithmeticException("mean of empty list!")
  else xs.sum / xs.length
```

Seq is the common interface of various linear sequence-like collections. Check the API docs (<http://mng.bz/f4k9>) for more information.

sum is defined as a method on Seq only if the elements of the sequence are numeric. The standard library accomplishes this trick with implicits, which we won't go into here.

The mean function is an example of what's called a *partial function*: it's not defined for some inputs. A function is typically partial because it makes some assumptions about its inputs that aren't implied by the input types.¹ You may be used to throwing exceptions in this case, but we have a few other options. Let's look at these for our mean example.

The first possibility is to return some sort of bogus value of type `Double`. We could simply return `xs.sum / xs.length` in all cases, and have it result in `0.0/0.0` when the input is empty, which is `Double.NaN`; or we could return some other sentinel value. In other situations, we might return `null` instead of a value of the needed type. This general class of approaches is how error handling is often done in languages without exceptions, and we reject this solution for a few reasons:

- It allows errors to silently propagate—the caller can forget to check this condition and won't be alerted by the compiler, which might result in subsequent code not working properly. Often the error won't be detected until much later in the code.
- Besides being error-prone, it results in a fair amount of boilerplate code at call sites, with explicit `if` statements to check whether the caller has received a “real” result. This boilerplate is magnified if you happen to be calling several functions, each of which uses error codes that must be checked and aggregated in some way.
- It's not applicable to polymorphic code. For some output types, we might not even *have* a sentinel value of that type even if we wanted to! Consider a function like `max`, which finds the maximum value in a sequence according to a custom comparison function: `def max[A](xs: Seq[A])(greater: (A,A) => Boolean): A`. If the input is empty, we can't invent a value of type `A`. Nor can `null` be used here, since `null` is only valid for non-primitive types, and `A` may in fact be a primitive like `Double` or `Int`.
- It demands a special policy or calling convention of callers—proper use of the mean function would require that callers do something other than call `mean` and make use of the result. Giving functions special policies like this makes it difficult to pass them to higher-order functions, which must treat all arguments uniformly.

The second possibility is to force the caller to supply an argument that tells us what to do in case we don't know how to handle the input:

```
def mean_1(xs: IndexedSeq[Double], onEmpty: Double): Double =
  if (xs.isEmpty) onEmpty
  else xs.sum / xs.length
```

This makes `mean` into a total function, but it has drawbacks—it requires that *immediate* callers have direct knowledge of how to handle the undefined case and limits them to

¹ A function may also be partial if it doesn't terminate for some inputs. We won't discuss this form of partiality here, since it's not a recoverable error so there's no question of how best to handle it. See the chapter notes for more about partiality.